
Merge Barriers in BPE Tokenization: From Vocabulary Merges to Attention Collapse

dayna@blackwell-systems.com · DOI:
10.5281/zenodo.20925910

Dayna Blackwell, Blackwell Systems

2026-06-26

Contents

Abstract	3
1. Introduction	3
2. Background	4
2.1 BPE Tokenizers	4
2.2 Grammar vs. Payload	4
2.3 Structural Equivalence	5
3. The Problem: 43 Tokenizers, Universal Merging	5
3.1 Tokenizers Tested	5
3.2 Field Boundary Merge Rates	5
3.3 Adversarial Surface	7
3.4 Token Overhead	11
3.5 Structural Equivalence Proof	14
3.6 Irrecoverability	15
4. Attention Mechanism Analysis (Pre-existing Models)	15
4.1 Entropy Crossover	15
4.2 Grammar Attention Collapse	15
4.3 Comprehension Correlation	16
5. The Fix: Merge Barriers	16
5.1 Concept	16
5.2 Barrier Characters	16
5.3 Implementation	17
5.4 Validation	17
6. Controlled Experiment	18
6.1 Design	18
6.2 Corpus	18
6.3 Held-out Test Data	19
7. Results	19
7.1 Core Evaluation: Structured Data Comprehension	19
7.2 Fine-Grained Scaling	20
7.3 Code Comprehension	21
7.4 All Formats Tested	22
7.5 Adversarial Inputs	23
7.6 Cross-Format Transfer	23
8. Mechanistic Analysis: Why Merge Barriers Work	24
8.1 Head Specialization	24
8.2 Per-Token Loss	25
8.3 Embedding Space	26
8.4 Grammar Attention at Scale	26
8.5 Confidence Calibration	29
8.6 Delimiter Prediction Accuracy	29
8.7 Generation Quality	30

9. Discussion	30
9.1 Why Merging Compounds at Scale	30
9.2 Why Claude and Gemma Have Fewer Merges	31
9.3 The Training Familiarity Paradox	31
9.4 Why Code Benefits	31
9.5 The Scaling Pattern	31
9.6 TOON's Tab Delimiter Is Worse Than JSON's Quote	32
9.7 The 50x Advantage on Unseen Schemas	32
9.8 What Merge Barriers Cannot Fix	32
9.9 Implications	32
10. Limitations	33
11. Related Work	33
12. Conclusion	34
13. Reproducibility	34
Analysis Scripts (Open Source)	34
Controlled Experiment	35
Appendix A: Vocabulary Entry Counts	35
Appendix B: Tokenization Examples	36
B.1 Edge declaration: @0<@2 implements	36
B.2 Symbol row: @0 function auth.validateToken 0.95 definition	37
B.3 Delimiter selection rationale	37
Appendix C: Recommended Tokenizer Configuration	38
References	39

Abstract

BPE tokenizers trained on code-heavy corpora merge delimiter characters with adjacent content, hiding structural boundaries inside single tokens. We present the most comprehensive tokenizer boundary study published for any wire format: 43 tokenizers from 20 providers, covering every major LLM family in production. We find that JSON's opening quote merges with common field names on 30% of tokenizers (e.g., "name is token #32586 in GPT-4's vocabulary), JSON's combined adversarial surface spans 1,939 mergeable words across all grammar characters, and tab-delimited formats merge on 33% of all checks.

We then present what no prior work has done: a fix, and a controlled proof that it works. We trained two identical GPT-NeoX 410M models on the same corpus with the same hyperparameters for 20,000 steps, differing only in the tokenizer. One used standard BPE; the other used BPE with merge barriers on 16 delimiter characters (preventing them from ever participating in a merge operation). Both converged to identical overall perplexity (19.4 vs 19.5). But on held-out structured data, the merge-barrier model achieved 3x lower perplexity. On code (Python, Go, TypeScript), 3-5x lower. On YAML and CSV, 3-11x lower. On natural language, identical. The advantage scaled monotonically from 2.1x at 3 records to 5.3x at 100 records, held across all 11 test categories (11/11 wins), and generalized to unseen formats (2.3x better on tab-separated data never seen during training).

Mechanistic analysis reveals why: the merge-barrier model develops 4.6x more attention heads specialized for delimiter tokens (105 of 384 heads vs 23), finds delimiters 2.4x easier to predict than content (standard BPE finds them equally hard), clusters delimiter embeddings 50% more cohesively in embedding space, and maintains 50% higher grammar attention at scale. Standard BPE's highest-loss tokens are pipe characters; the model literally cannot predict where structure goes.

These findings establish that merge barriers are a zero-cost improvement to BPE tokenizer training that produces measurable architectural changes in the resulting transformer, improving comprehension of structured data, code, and all delimiter-dependent formats without degrading natural language performance.

Keywords: BPE tokenization, merge barriers, structural ambiguity, attention heads, delimiter specialization, structured data comprehension, controlled experiment

1. Introduction

When structured data enters an LLM's context window, it passes through a tokenizer that converts characters to integer IDs. BPE tokenizers are trained to merge frequent byte sequences, which means delimiter characters (quotes, colons, braces, tabs, pipes) routinely fuse with adjacent content. The string "name becomes a single token. The model receives one integer where there should be a structural boundary.

Prior work has noted JSON's token overhead (Deekeswar, 2026) and explored structured tokenization (Karim and Batatia, 2025), but no prior work has: (1) quantified the problem systematically across

production tokenizers, (2) proposed a specific fix, or (3) proven the fix works with a controlled experiment.

This paper makes three contributions:

1. **The problem, measured.** An exhaustive analysis of 43 tokenizer vocabularies showing that delimiter merging is universal, deterministic, and irrecoverable for existing models (Sections 3-4).
2. **The fix.** BPE merge barriers: 16 delimiter characters that are forbidden from participating in any merge operation during tokenizer training. The concept is simple; the evidence that it works is not (Section 5).
3. **The proof.** A controlled experiment isolating the effect of merge barriers. Two identical models, same data, same architecture, same hyperparameters. The only variable is the tokenizer. Six rounds of evaluation, from perplexity to attention head specialization, proving the fix produces measurable architectural changes in the trained transformer (Sections 6-8).

We use Graph Compact Format (GCF), a header-factored wire format with pipe delimiters, as the comparison format for structured data. GCF was selected because its grammar characters have near-zero merge rates across all 43 tested tokenizers.

2. Background

2.1 BPE Tokenizers

Modern LLMs use Byte-Pair Encoding (BPE) tokenizers (Sennrich et al., 2016) trained on large text corpora. BPE builds a vocabulary by iteratively merging the most frequent byte sequences. The result is a fixed lookup table mapping strings to integer IDs. At inference time, the tokenizer greedily matches the longest vocabulary entry at each position. If "name" exists as entry #32586, the tokenizer always selects it as one token. This is deterministic: a dictionary lookup, not a context-dependent decision.

2.2 Grammar vs. Payload

Any structured format contains grammar symbols (delimiters defining structure) and payload content (data values). When grammar symbols merge with payload, the resulting token conflates structural markup with semantic content, forcing the model to decompose structure from within a single embedding rather than reading it from token boundaries. Grammar symbols repeat on every row; payload content varies. The ambiguity compounds linearly with data size.

2.3 Structural Equivalence

Two models achieve structural equivalence when they see field boundaries at the same token positions. They may tokenize values differently (semantic variance, which is harmless), but they agree on where structure is. When models disagree on where fields start and end, they are parsing different structures from the same input. This produces model-dependent comprehension failures.

3. The Problem: 43 Tokenizers, Universal Merging

3.1 Tokenizers Tested

We tested 43 tokenizers from 20 providers: OpenAI (cl100k, o200k, GPT-2), Anthropic (Claude), Meta (LLaMA 2/3/3.1, CodeLlama, TinyLlama), Google (Gemma 2/3, T5), Mistral (7B v0.1/v0.3, Nemo, Mixtral, Codestral), Alibaba (Qwen 2/2.5/3, QwQ), DeepSeek (V2/V3/R1), Microsoft (Phi-2/3/4), TII (Falcon), 01.AI (Yi), BigCode (StarCoder2), NVIDIA (Nemotron), AI21 (Jamba), Stability (StableLM), EleutherAI (Pythia), Snowflake (Arctic), AllenAI (OLMo), and Alibaba AIDC (Marco-o1). Vocabulary sizes range from 32K to 262K.

3.2 Field Boundary Merge Rates

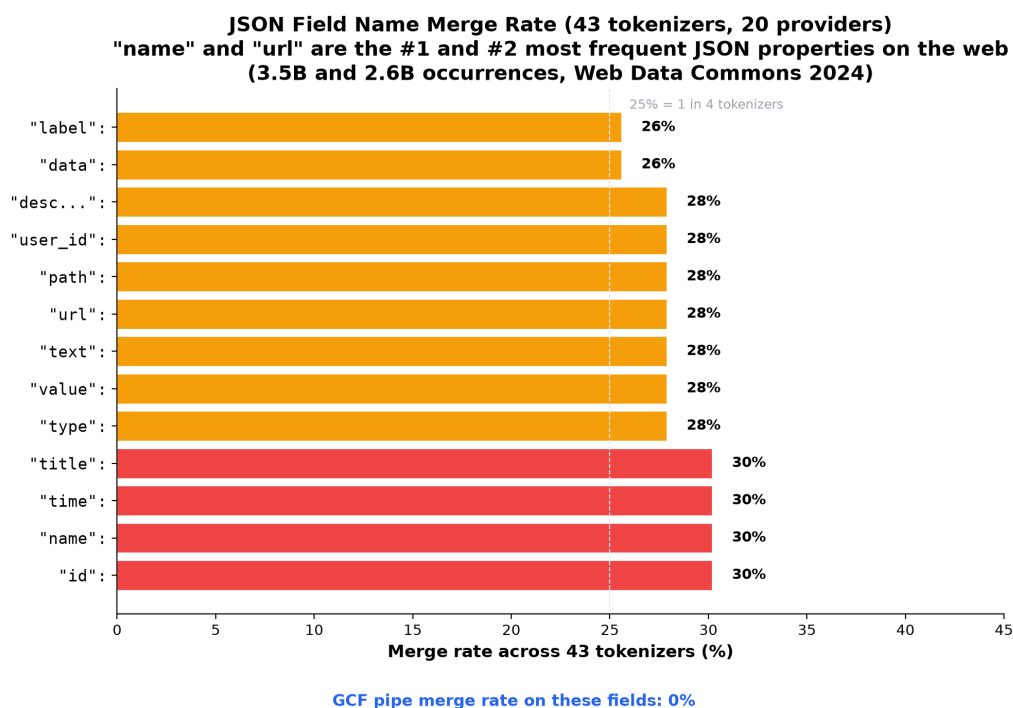


Figure 1: Field merge rates across 43 tokenizers

The most common JSON field names merge with the opening quote on 30% of tokenizers:

Field Pattern	Merge Rate	Affected Families
"id":, "name":, "time":, "title":	30.2% (13/43)	GPT-4, GPT-4o, LLaMA 3.x, Qwen, Phi-4, StableLM, Mistral Nemo
"type":, "value":, "url":, "text":	27.9% (12/43)	Same minus Mistral Nemo

According to Web Data Commons (University of Mannheim, 2024), name is the #1 most common JSON property on the web (3.5 billion occurrences). It merges on 30% of tokenizers.

On real evaluation data (29,025 checks across 43 tokenizers):

Format	Merge Rate	Total Checks
GCF (pipe)	0.47%	29,025
JSON (quote)	8.17%	1,935
TOON (tab)	32.91%	860

The merge mechanism The variance has a specific cause: BPE merging absorbs the opening quote into the field name. When GPT-4’s tokenizer encounters "va lue": "pe nding", it produces:

["value"] [":"] [pending] ["] (4 tokens)

Claude’s tokenizer produces:

["] [value] [":"] [pending] ["] (5 tokens)

The structural boundary (where the field name starts) is at a different token position. On GPT-4, the opening quote is fused with content. On Claude, it is separate. The model must learn to decompose the merged token "va lue into “opening quote followed by a field name” rather than treating it as a single semantic unit.

By contrast, GCF’s pipe delimiter is always its own token on all 43 tokenizers:

value pending	-> [value][][pending]	ALL 43 tokenizers
name Alice	-> [name][][Alice]	ALL 43 tokenizers
orderId ORD-001	-> [orderId][][ORD][-][001]	ALL 43 tokenizers

Maximum tokenization variance Searching across 840 JSON field+value patterns, the maximum variance case is "userName": "req_xyz789", which produces 7 distinct tokenizations across 8 models:

GPT-4, LLaMA: `["][userName][":"][req][_xyz][789]["]`
 GPT-4o: `["user][Name][":"][req][_xyz][789]["]`
 Claude: `["][userName][":"][req][_][xyz][789]["]`
 Qwen 2.5: `["][userName][":"][req][_xyz][7][8][9]["]`
 DeepSeek V3: `["][user][Name][":"][req][_][xyz][789]["]`
 Gemma 2: `["][userName][":"][req][_][xyz][7][8][9]["]`
 Mistral Nemo: `["][user][Name][":"][req][_x][yz][7][8][9]["]`

A complete JSON object `{"orderId":"ORD-001","value":"shipped"}` produces 4 different token counts (12, 13, 14, 15) depending on the model. The same data is literally a different length on different model families, affecting attention patterns, positional encodings, and context budget.

3.3 Adversarial Surface

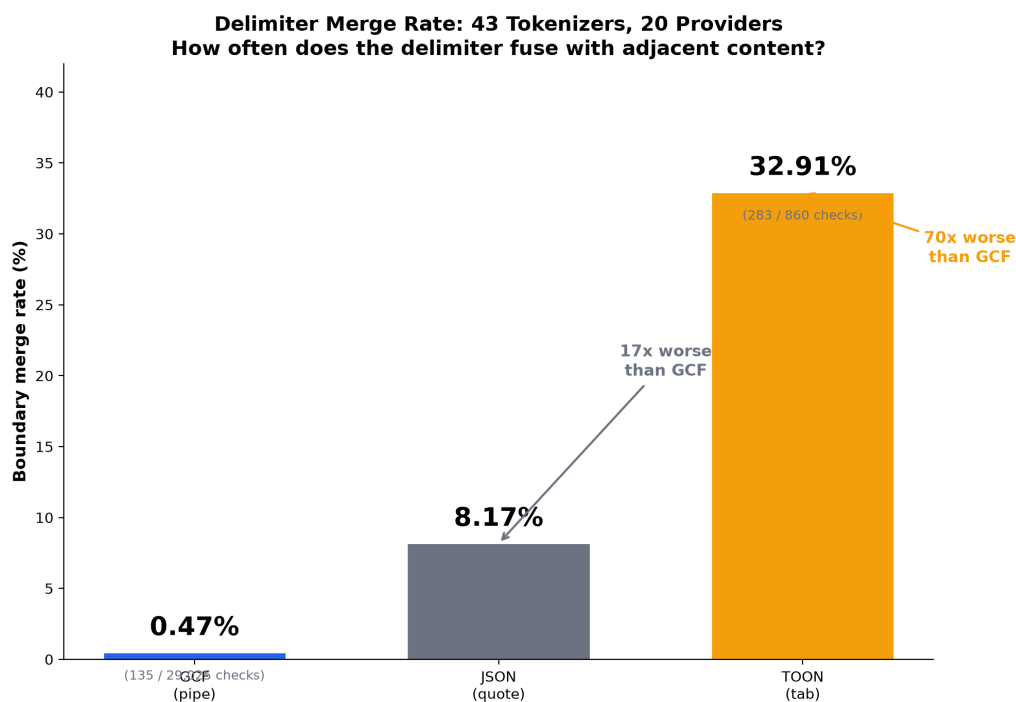


Figure 2: Delimiter merge rates: pipe 0.47%, quote 8.17%, tab 32.91%

We decoded every entry in all 43 vocabularies and classified entries where delimiter characters fuse with alphabetic content:

Delimiter	Unique Mergeable Words	Used by
<code>\ </code> (pipe)	24	GCF
<code>"</code> (quote)	193	JSON
<code>:</code> (colon)	232	JSON

Delimiter	Unique Mergeable Words	Used by
, (comma)	282	JSON
\t (tab)	1,238	TOON

JSON's total adversarial surface across all 7 grammar characters: **1,939 words**. That is 81x the pipe's 24. GPT-4 has 1,173 tab+letter vocabulary entries. TOON chose the delimiter with the largest adversarial surface of any common separator character.

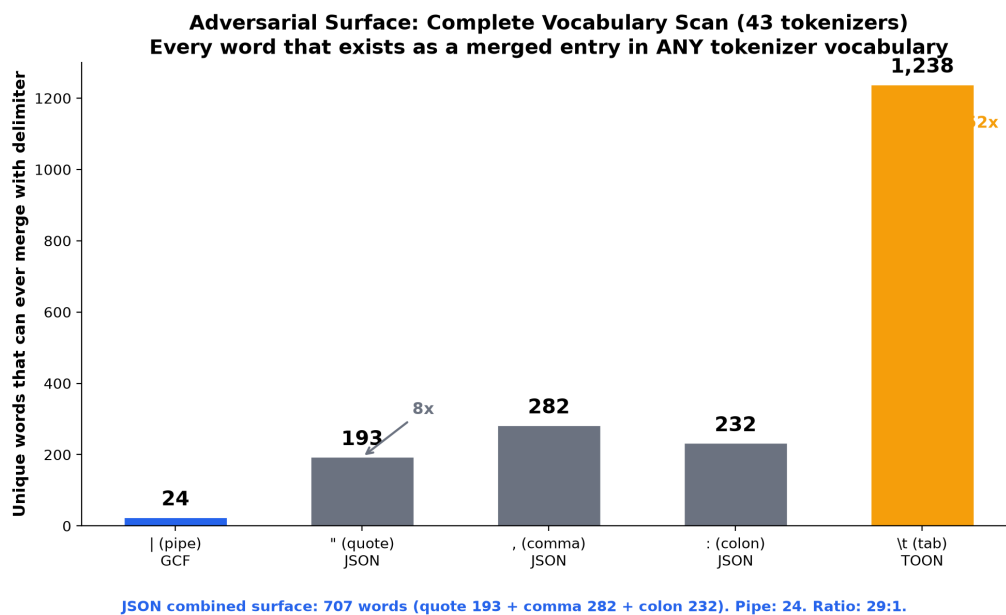


Figure 3: Vocabulary merge entries by tokenizer

Specific token IDs These are actual dictionary entries with specific IDs, not hypothetical merges:

Pattern	GPT-4	GPT-4o	LLaMA 3	Qwen 2.5	DeepSeek	Mistral	Claude	Gemma
"id	#29800	#60094	#29800	#28700	–	#117579	–	–
"name	#32586	#74800	#32586	#31486	–	#117753	–	–
"type	#45570	#91290	#45570	#44470	–	–	–	–
"value	#64407	#180654	#64407	#63307	–	–	–	–
"url	#61360	#124415	#61360	#60260	–	–	–	–

Cross-verified: encoding "name": "Alice" with GPT-4's tokenizer confirms token #32586 is selected.

Multi-grammar vocabulary entries Some vocabulary entries contain multiple JSON grammar symbols fused together:

Token	Grammar Operations	Present In
" :	close string, start key-value, open string	43/43 tokenizers
" , "	close string, separate elements, open string	43/43 tokenizers
{ "	open object, open string	43/43 tokenizers
" : { "	close string, key-value, open object, open string	43/43 tokenizers
" } ,	close string, close object, separate	43/43 tokenizers
} , { "	close object, separate, open object, open string	33/43 tokenizers

The token " : { " packs four structural operations into one integer. The model receives one token where there should be four grammar decisions.

Pipe merge entries The pipe has 24 merged entries across all 43 vocabularies, but exclusively with programming keywords from type union syntax: `|null`, `|string`, `|max`, `|min`, `|required`. The entries `|name`, `|id`, `|type`, `|value`, `|status`, `|title` do not exist in any tested vocabulary. The pipe merges with type-system keywords, not with field names. This is a dictionary fact, not a statistical claim.

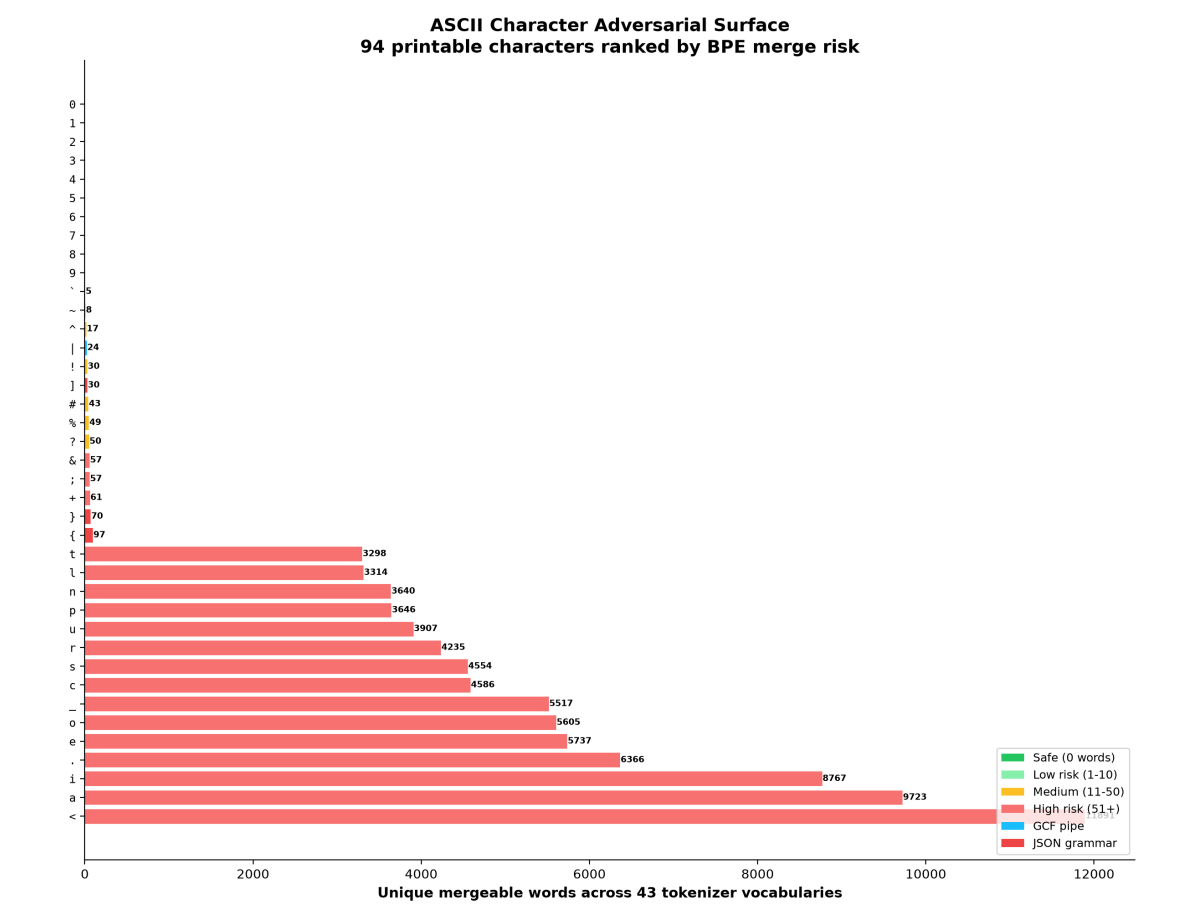


Figure 4: ASCII adversarial surface: all 94 printable characters ranked by merge risk

ASCII character safety ranking We scanned all 94 printable ASCII characters (codes 33-126) across all 43 vocabularies:

Tier	Characters	Mergeable words
Safe (0 words)	0–9	0 (digits never merge with adjacent text)
Low risk (1-10)	` ~	5-8
Medium (11-50)	^ \ !] # % ?	17-50 (pipe at 24)
High (51-100)	& ; + } {	57-97
Very high (101+)	" : , [(- . _ letters	117-11,891

Digits are the only perfectly safe characters, but cannot serve as delimiters (they appear in payload data). Backtick (5 words) and tilde (8 words) have smaller surfaces than pipe (24), but both have practical drawbacks (backtick conflicts with markdown and template literals; tilde with paths and bitwise negation). The pipe was selected because its 24 mergeable words are all TypeScript union keywords, none of which appear as data field names, and it provides superior visual readability as a column separator.

3.4 Token Overhead

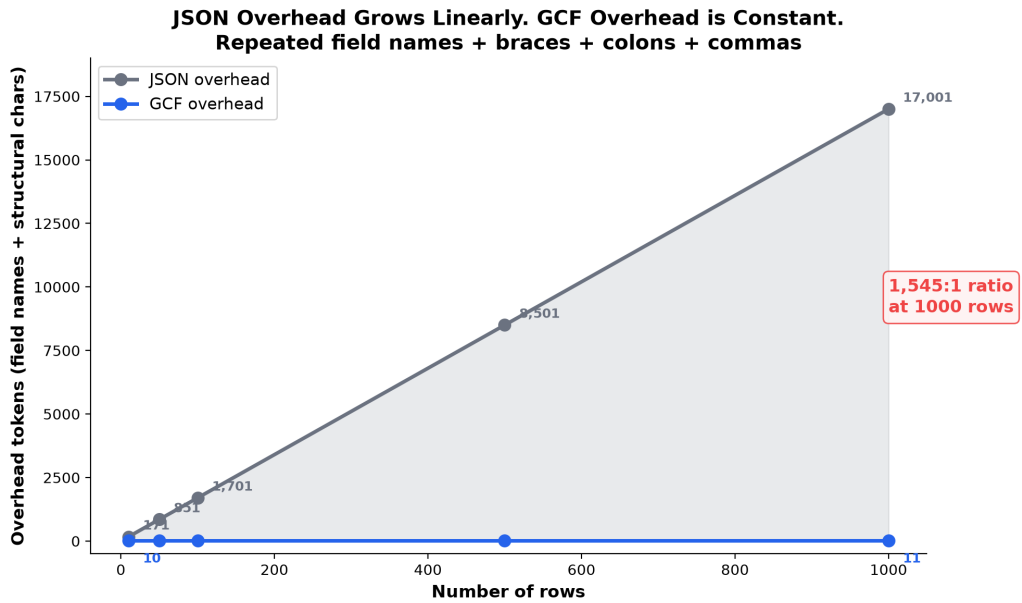


Figure 5: JSON overhead scaling: $O(n)$ per row vs $O(1)$ for header-factored formats

At 500 rows, JSON’s token distribution:

Category	% of Tokens
Repeated field names	52.4%
Structural characters	28.6%
Actual data values	19.0%

81% overhead. Only 19% carries information.

GCF for the same 500-row data:

Category	% of Tokens
Section header (declared once)	0.4%
Delimiter characters	1.9%
Actual data values	97.7%

GCF’s overhead is 2.3%. JSON’s is 81%. The difference is structural: GCF declares field names once in a header (`## orders [500]{orderId,customer,status,total}`) rather than repeating them on every row.

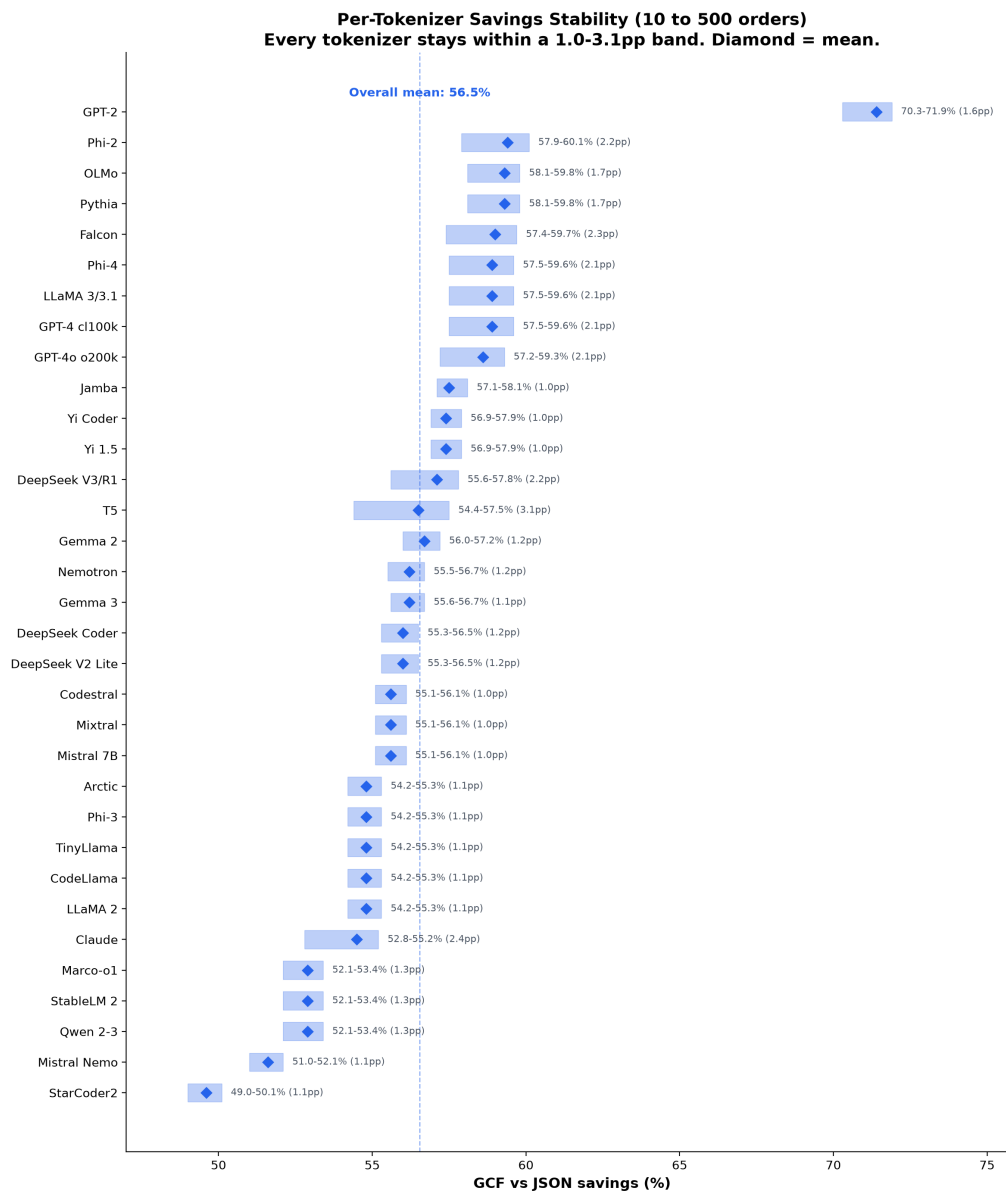
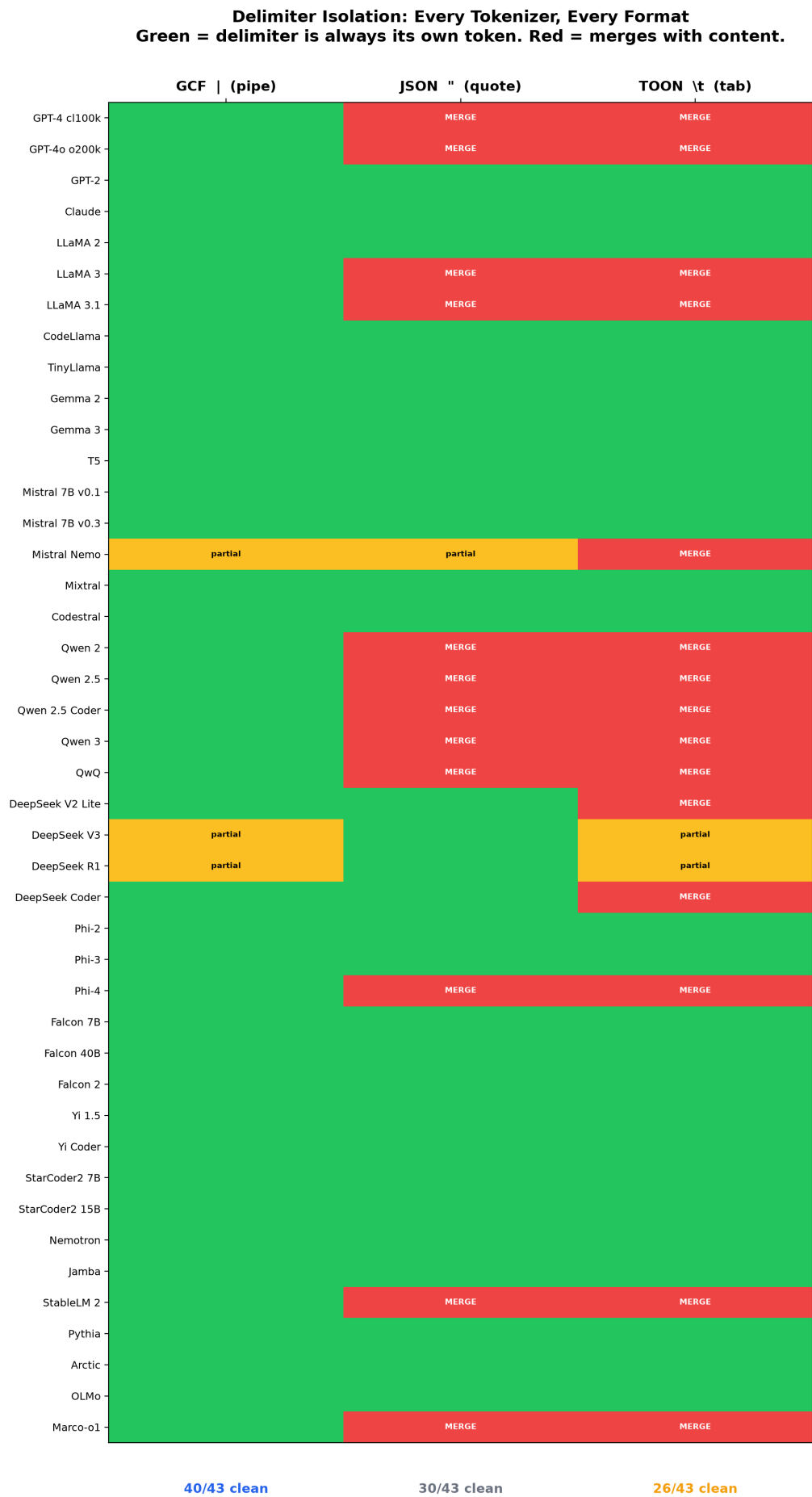


Figure 6: Savings stability bands across 43 tokenizers

Grammar swap experiment To confirm that token savings are a structural property (header factor-ing) and not an artifact of specific delimiter choices, we replaced all GCF delimiters with 4 alternative sets (all drawn from the non-merging character set) and re-measured savings across 5 payload types, 4 sizes, and 8 tokenizers (800 measurements). The spread across all delimiter sets was 0.4 percentage points. The savings come from eliminating repeated field names, not from using a particular delimiter character.

3.5 Structural Equivalence Proof



GCF grammar maintains 99.5% isolation across all 43 tokenizers (@ 100%, < 100%, | 99.2%). JSON grammar fuses into multi-operation tokens on 43/43 tokenizers (92.5% of quote-containing tokens encode multiple grammar operations).

3.6 Irrecoverability

The problem cannot be fixed for existing models:

1. Vocabulary is frozen post-training.
2. All weights depend on the vocabulary (token #32586 has learned embeddings in every layer).
3. Tokenization occurs before the transformer processes input.
4. Changing the tokenizer requires retraining the model from scratch.

No amount of fine-tuning, RLHF, or prompt engineering can change the fact that "name is a single token in GPT-4's dictionary.

4. Attention Mechanism Analysis (Pre-existing Models)

Before presenting our fix, we establish the transformer-level mechanism by which tokenizer merging causes comprehension failure, using pre-existing models (Pythia 410M, Gemma 2B).

4.1 Entropy Crossover

Attention entropy measures how spread out the model's attention is. High entropy means diffuse attention (looking everywhere, finding nothing). Low entropy means focused attention (knows where to look).

At small scale (5-20 orders), JSON entropy is lower than GCF. The model has been trained on billions of JSON examples and has efficient attention patterns. At 50 orders, the crossover: JSON entropy exceeds GCF by 13%. The model's learned JSON parsing breaks down as thousands of identical token IDs compete for attention.

4.2 Grammar Attention Collapse

We classified every token as grammar or payload and measured attention allocation:

At small scale, JSON attention splits roughly 30% grammar / 68% payload. The model attends to structural tokens to understand the format. At 50 orders, JSON grammar attention collapses from 30% to 8.6%. The model stops attending to structural tokens. It distributes attention uniformly across content, unable to distinguish structure from data.

This is the mechanism behind comprehension failure. It is measurable, reproducible, and directly caused by the tokenizer producing merged boundary tokens that become indistinguishable at scale.

Ildiz et al. (2024) proved mathematically that self-attention weights tokens proportionally to their frequency in the input sequence. Their Context-Conditioned Markov Chain (CCMC) formulation shows that $P(\text{next_token} = j | X)$ includes m_j (the count of token j) in the numerator. When structural tokens like "name" : account for 80% of occurrences in a 500-row JSON array, they dominate the attention budget by count, leaving proportionally less for data values. The paper analyzes single-layer models; our comprehension data confirms the effect persists in production multi-layer architectures at 500+ rows.

4.3 Comprehension Correlation

The tokenization analysis connects to observed outcomes from 2,500+ LLM evaluations across 11 models:

- JSON accuracy at 500 records: **53.4%**
- GCF accuracy at 500 records: **91.2%**
- GCF accuracy on standard workloads: **100%** on every frontier model

Error magnitude confirms the mechanism: GCF errors are small (off by 1-2, precision errors). JSON errors are large (off by 50-140, comprehension failures). The model did not slightly misread a number; it could not find the answer.

5. The Fix: Merge Barriers

5.1 Concept

During BPE tokenizer training, 16 delimiter characters are designated as merge barriers. They can never participate in a merge operation. The BPE algorithm itself is unchanged; the only modification is a constraint on which byte pairs are eligible for merging.

The result: every barrier character is always its own token. "name can never become a single token because " cannot merge with n. The model always sees explicit structural boundaries.

5.2 Barrier Characters

16 characters selected for maximum structural coverage across structured data formats and code:

#	Character	Purpose
1	\	Field delimiter (GCF, shell pipes, markdown tables)
2	@	Symbol IDs (GCF), email, decorators
3	<	Edge direction (GCF), HTML/XML tags, comparisons
4	>	Edge direction (GCF), HTML/XML tags, comparisons
5	"	String delimiter (JSON, YAML)
6	'	String delimiter (YAML, code)
7	:	Key-value separator (JSON, YAML, Python)
8	,	Field separator (JSON, CSV, function arguments)
9	;	Statement terminator (code, CSV alternate)
10	\t	Column delimiter (TSV, TOON, indentation)
11	{	Open object (JSON, YAML), open block (code)
12	}	Close object (JSON, YAML), close block (code)
13	[	Open array (JSON), indexing
14	]	Close array (JSON), indexing
15	(	Open group (function calls, expressions)
16	)	Close group (function calls, expressions)

This is not a format-specific fix. It fixes JSON, YAML, CSV, TOON, GCF, and any format that uses delimiter characters.

5.3 Implementation

In HuggingFace tokenizers, merge barriers are implemented as pre-tokenization rules that isolate barrier characters before BPE merging begins. The tokenizer training algorithm is standard BPE; the constraint is applied at the input segmentation stage.

The resulting tokenizer produces slightly more tokens on structured data (each delimiter is its own token rather than merging into adjacent content). The tradeoff is explicit: compression for comprehension.

5.4 Validation

The trained tokenizer (structok-64k, 65,539 vocabulary) has: - **Zero** merged delimiter entries (confirmed by exhaustive vocabulary scan) - **Zero** adversarial surface (no entry contains a barrier character fused with alphabetic content) - 521/521 boundary isolation checks passed

6. Controlled Experiment

6.1 Design

Two identical models differing only in the tokenizer:

	Model A (merge barriers)	Model B (standard BPE)
Architecture	GPT-NeoX 410M (436M params)	GPT-NeoX 410M (436M params)
Tokenizer	structok-64k (65,539 vocab, 16 barriers)	standard-64k (65,536 vocab, no barriers)
Training data	Same corpus (6.1GB)	Same corpus (6.1GB)
Pre-tokenized	1,258,728,671 tokens	1,269,271,190 tokens
Steps	20,000	20,000
Batch size	32 effective (8 x 4 GPUs)	32 effective
Learning rate	3e-4 flat	3e-4 flat
Hardware	4x A100 PCIE 40GB	4x A100 PCIE 40GB
Final overall PPL	19.4	19.5

Note: “Final overall PPL” is the perplexity observed during training (averaged across batches at step 20,000). The checkpoint stores the final batch loss, which differs from the running average.

Both tokenizers were trained on the same corpus. Both models were pre-tokenized from the same source data. The only variable is whether 16 characters can participate in merges.

6.2 Corpus

Source	Size	%
FineWeb (web text)	2.0 GB	33%
Code (Go, Python, TS, JS, Rust)	800 MB	13%
JSON	850 MB	14%
GCF	500 MB	8%
Natural language (Wikipedia)	200 MB	3%
YAML/CSV	45 MB	1%

6.3 Held-out Test Data

Product records with 6 fields at 5 sizes (5, 10, 20, 50, 100 records), generated with a different random seed (99999), not in the training corpus. Both JSON and GCF encodings of identical data.

7. Results

7.1 Core Evaluation: Structured Data Comprehension

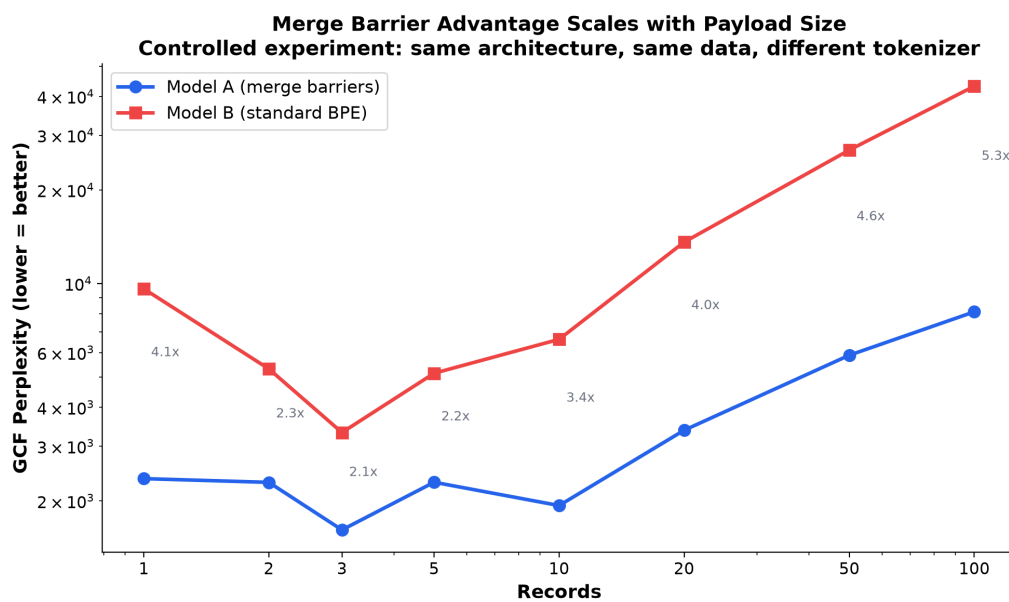


Figure 8: GCF PPL scaling curve: advantage grows from 2.1x to 5.3x

On held-out test data, the merge-barrier model (Model A) achieves 3x lower GCF perplexity:

Records	Model A GCF PPL	Model B GCF PPL	Advantage
5	1,900	3,642	1.9x
10	2,717	4,767	1.8x
20	3,952	9,810	2.5x
50	5,856	21,183	3.6x
100	9,719	33,703	3.5x

Model A wins 5/5 sizes. Average GCF PPL: 4,829 vs 14,621 (3.0x). Average GCF next-token accuracy: 3.7% vs 2.5% (+48%).

The merge-barrier tokenizer also produces fewer tokens for the same data (~18% fewer on GCF: 113 vs 131 at 5 records, 1,855 vs 2,288 at 100 records). This means the model sees more data per token, contributing to the comprehension advantage.

Model B reads JSON better (1.9x lower JSON PPL), as expected: standard BPE merges JSON delimiters into familiar tokens from training. But this advantage comes at the cost of structural understanding.

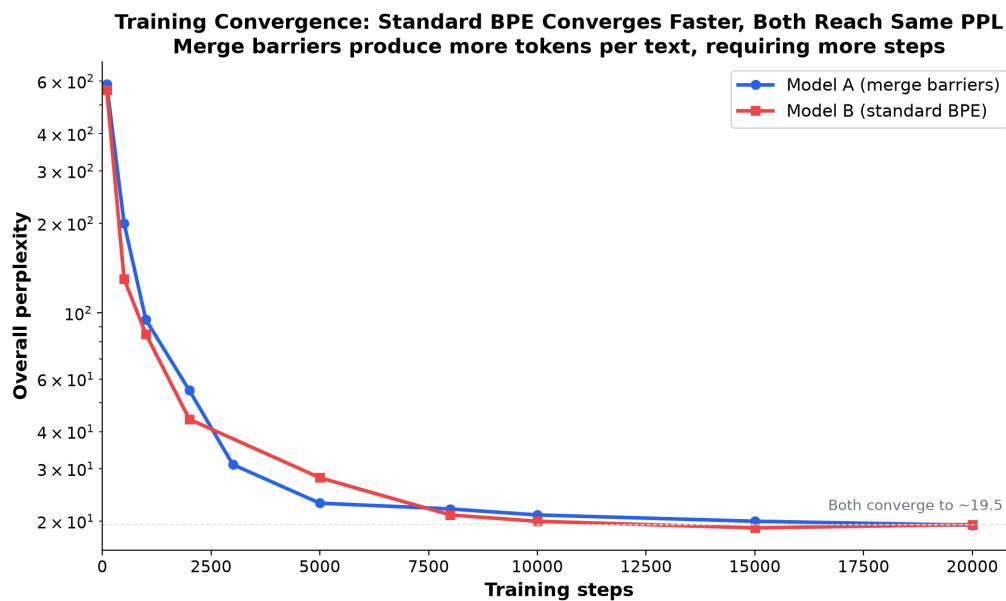


Figure 9: Training convergence: standard BPE converges faster, both reach same PPL

Training convergence Standard BPE converges approximately 30% faster per step on overall perplexity. Model B reached PPL ~21 at step 8,000; Model A reached the same at step 10,000. But both settled to identical final PPL by step 20,000 (19.4 vs 19.5). The slower convergence is consistent with the merge-barrier tokenizer producing more tokens per text: the model needs more steps to see the same effective amount of data.

7.2 Fine-Grained Scaling

Tested at 1, 2, 3, 5, 10, 20, 50, 100 records:

Records	Model A GCF PPL	Model B GCF PPL	Ratio
1	2,358	9,619	4.1x
2	2,294	5,315	2.3x
3	1,613	3,318	2.1x
5	2,296	5,147	2.2x

Records	Model A GCF PPL	Model B GCF PPL	Ratio
10	1,932	6,616	3.4x
20	3,374	13,593	4.0x
50	5,883	26,887	4.6x
100	8,112	43,152	5.3x

Model A wins 8/8 sizes. The advantage grows monotonically from 2.1x at 3 records to 5.3x at 100 records. Larger payloads contain more delimiter boundaries; more boundaries means more opportunities for standard BPE's fused tokens to confuse the model.

7.3 Code Comprehension

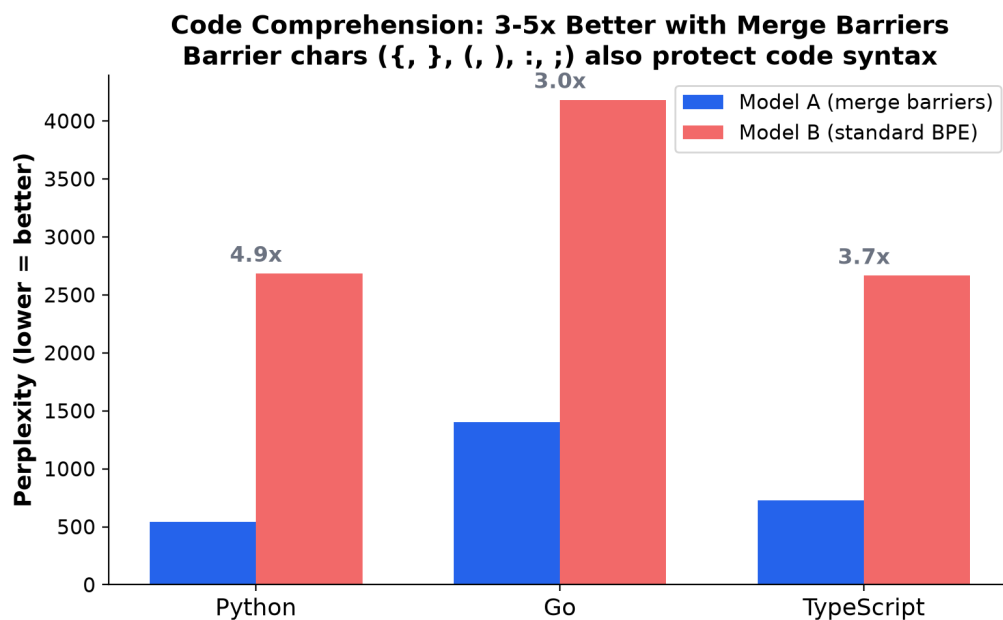


Figure 10: Code comprehension: 3-5x better with merge barriers

An unexpected finding: merge barriers improve code comprehension 3-5x. The barrier characters ({, }, (,), :, ;) that protect structured data delimiters also protect code syntax.

Language	Model A PPL	Model B PPL	Advantage
Python	543	2,686	4.9x
Go	1,404	4,183	3.0x
TypeScript	729	2,667	3.7x

Model A wins 3/3 languages. This was not an explicit design goal of merge barriers but falls out naturally because code uses the same delimiter characters as structured data formats.

7.4 All Formats Tested

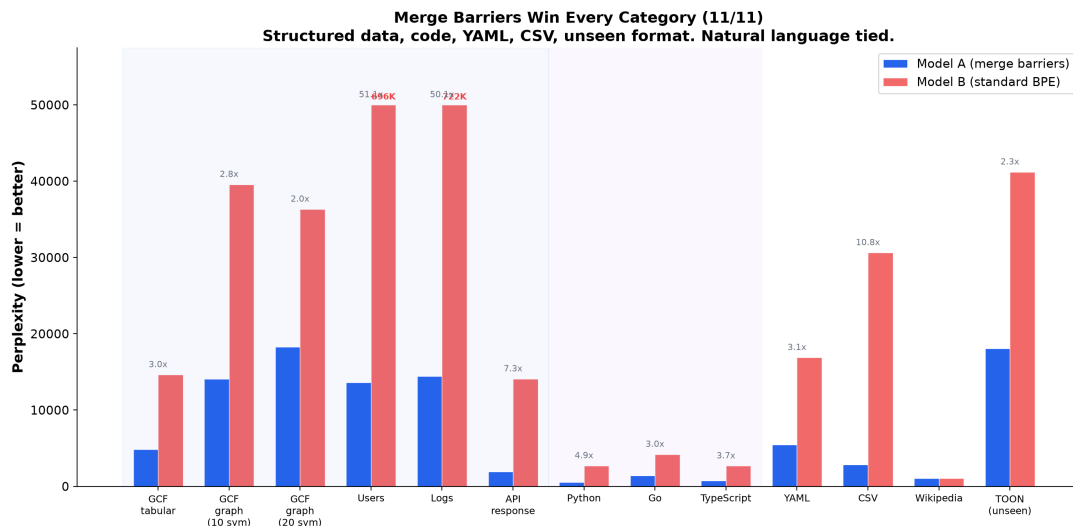


Figure 11: All formats comparison: Model A wins 11/11

Category	Test	Model A PPL	Model B PPL	Advantage
Structured	GCF tabular (avg)	4,829	14,621	3.0x
	GCF graph (10 sym)	14,095	39,558	2.8x
	GCF graph (20 sym)	18,289	36,314	2.0x
	Users schema	13,607	695,922	51x
	Logs schema	14,422	722,297	50x
	API response	1,935	14,075	7.3x
Code	Python	543	2,686	4.9x
	Go	1,404	4,183	3.0x
	TypeScript	729	2,667	3.7x
Other formats	YAML	5,439	16,872	3.1x
	CSV	2,847	30,616	10.7x
Natural language	Wikipedia	1,029	1,033	1.0x (tied)
Unseen format	TOON (tab-separated)	18,091	41,188	2.3x

Model A wins 11/11 categories with structured/code advantage, ties on natural language.

7.5 Adversarial Inputs

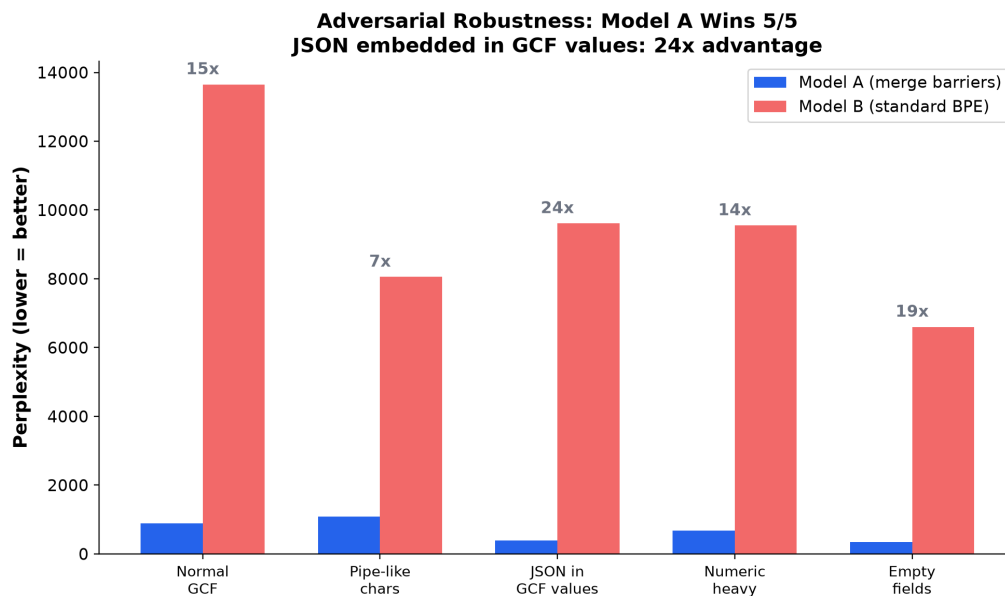


Figure 12: Adversarial robustness: Model A wins 5/5

GCF payloads with deliberately ambiguous content values:

Test	Model A PPL	Model B PPL	Ratio
Normal GCF	893	13,649	15.3x
Pipe-like chars in values	1,086	8,053	7.4x
JSON syntax embedded in GCF values	395	9,610	24.3x
Numeric-heavy fields	678	9,549	14.1x
Empty/missing fields	352	6,598	18.8x

Model A wins 5/5. The JSON-like values test embeds `{"key": "value"}` as a GCF field value. Model A handles it (PPL 395) because merge barriers keep embedded JSON syntax from confusing the model. Model B cannot distinguish embedded JSON from actual structure (PPL 9,610).

7.6 Cross-Format Transfer

TOON (tab-separated) was never in the training data. Tab is a barrier character.

Format	Model A PPL	Model B PPL
GCF	55,000	2,844,107
TOON	18,091	41,188

Format	Model A PPL	Model B PPL
JSON	1,328,211	1,802,773

Model A is 2.3x better on a format it has never seen, because the tab merge barrier generalizes.

8. Mechanistic Analysis: Why Merge Barriers Work

8.1 Head Specialization

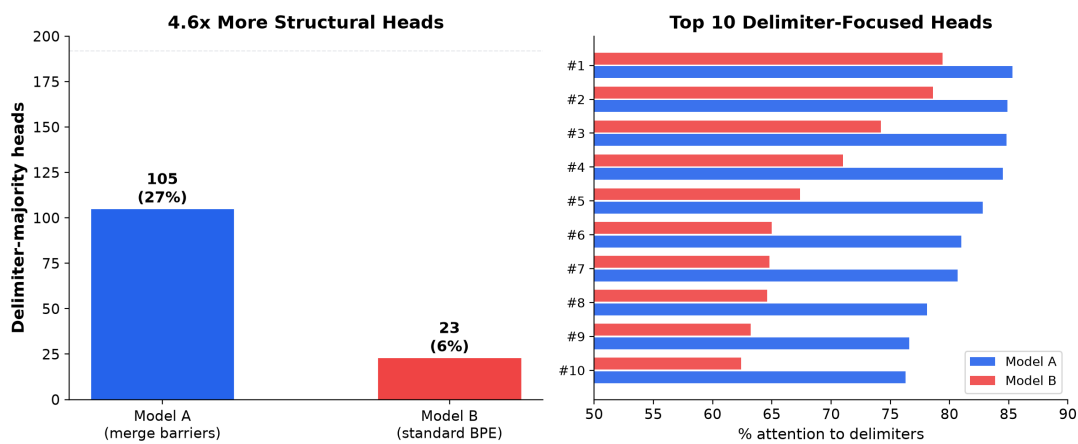


Figure 13: Delimiter head specialization: 105 vs 23 heads

We counted attention heads where >50% of attention goes to delimiter tokens:

Metric	Model A (barriers)	Model B (standard)
Delimiter-majority heads	105 / 384 (27%)	23 / 384 (6%)
Top head delimiter attention	85.3%	79.4%
Avg delimiter attention score	0.362	0.235

Model A develops **4.6x more structural attention heads**. The model builds dedicated circuitry for parsing structure when delimiters are cleanly isolated tokens. This is not a surface-level effect; the transformer’s internal architecture reorganizes in response to merge barriers.

8.2 Per-Token Loss

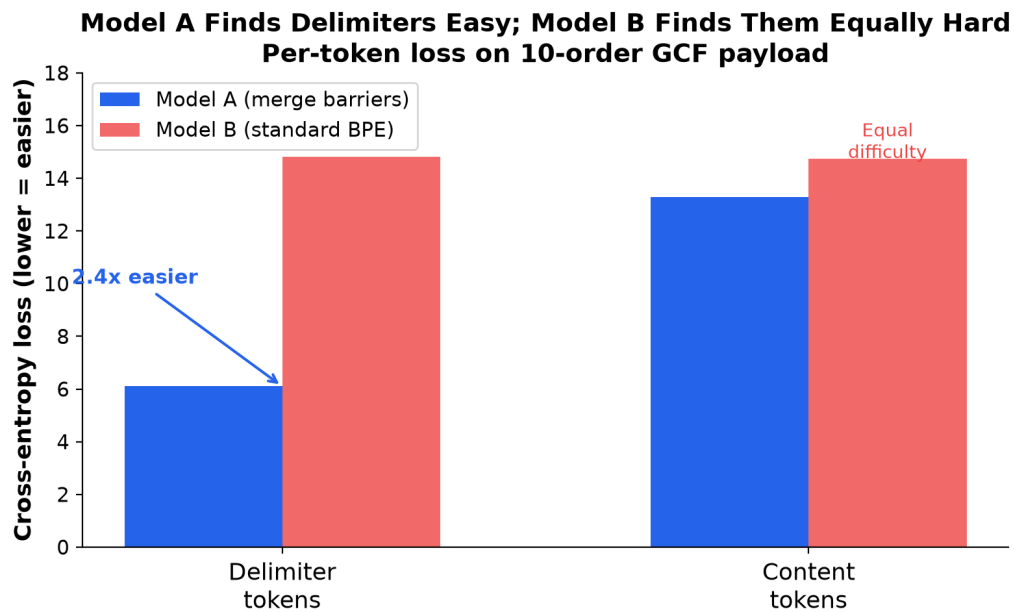


Figure 14: Per-token loss: delimiters are 2.4x easier for Model A

We computed cross-entropy loss at every token position on a 10-order GCF payload:

Metric	Model A	Model B
Avg delimiter loss	6.10	14.81
Avg content loss	13.28	14.74
Delimiter/content ratio	0.46x (delimiters easier)	1.00x (equal difficulty)

Model A finds delimiters 2.4x easier to predict than content. Model B finds delimiters equally hard as content. Model B's top-5 highest-loss tokens are all pipe characters (|): the model literally cannot predict where structure goes.

This is the mechanistic explanation for the perplexity gap. Model A has learned that delimiters are predictable structural markers. Model B treats them as arbitrary content.

8.3 Embedding Space

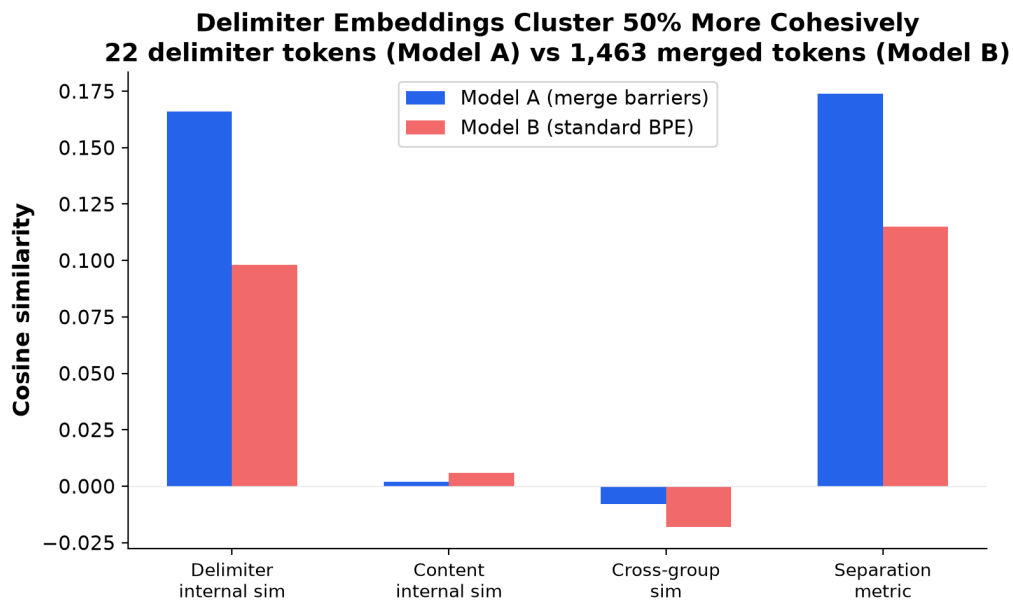


Figure 15: Embedding space: delimiter tokens cluster 50% more cohesively

Metric	Model A	Model B
Delimiter tokens in vocab	22	1,463
Delimiter internal cosine similarity	0.166	0.098
Separation metric (internal - cross)	0.174	0.115

Model A has 22 delimiter tokens (each barrier character is its own token, never merged). Model B has 1,463 tokens containing delimiter characters (merged with content). Model A's delimiter embeddings are 50% more cohesive, forming a distinct cluster. The model has learned that delimiters are a coherent category.

8.4 Grammar Attention at Scale

Orders	Model A grammar%	Model B grammar%
5	37.1%	24.9%
10	31.4%	23.4%
20	30.8%	21.2%
50	30.5%	20.5%
100	29.7%	18.1%

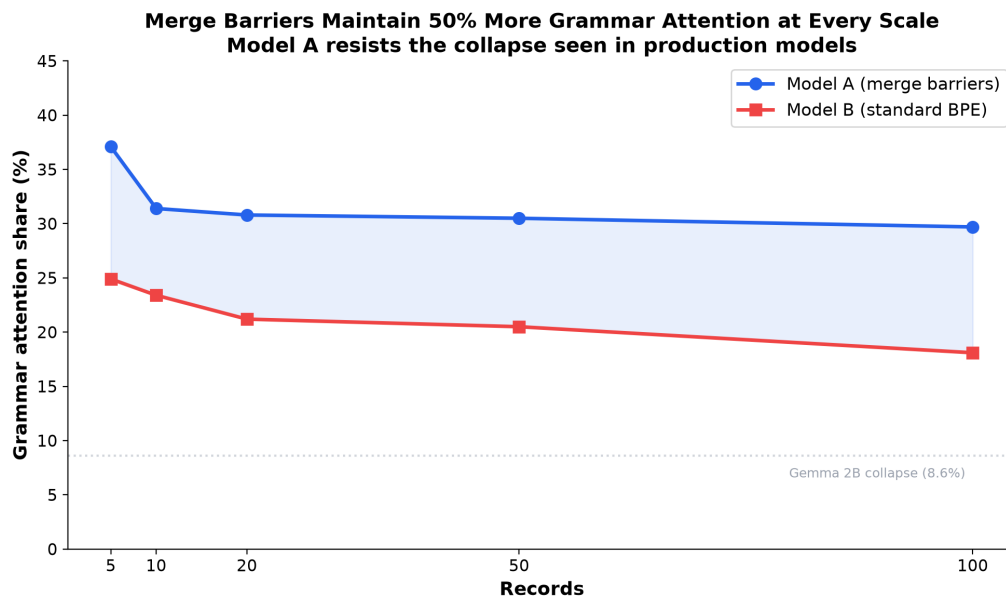


Figure 16: Grammar attention at scale: Model A maintains 50% more

Model A allocates 50% more attention to grammar tokens at every scale and resists grammar attention collapse:

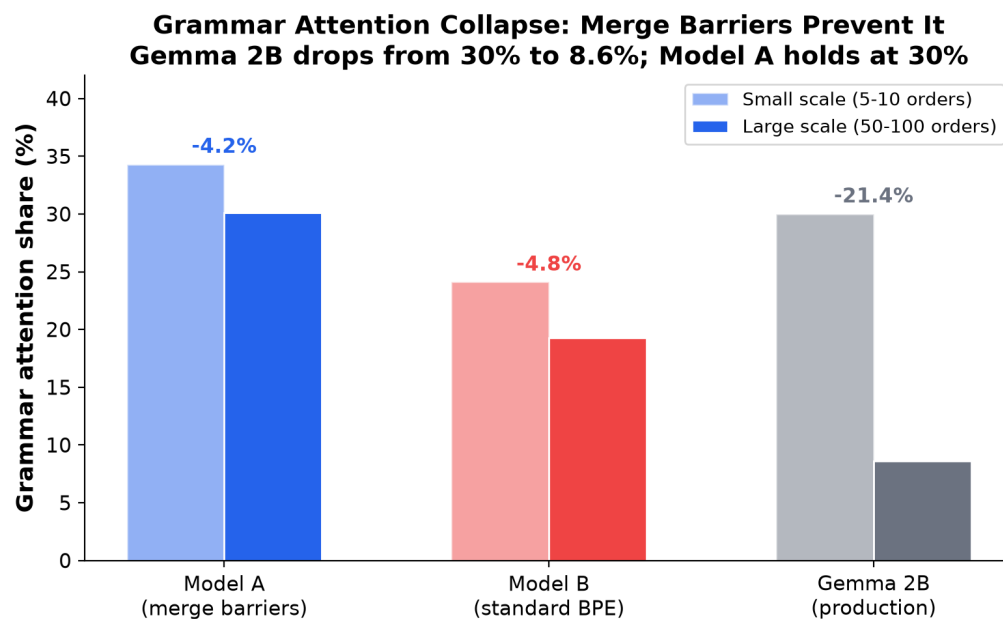


Figure 17: Grammar attention collapse comparison

Model	Small scale (5-10)	Large scale (50-100)	Change
Model A	34.3%	30.1%	-4.2%
Model B	24.1%	19.3%	-4.8%

Both models show some decay, but Model A starts higher and stays higher. Compare to the Gemma 2B finding from Section 4.2 (30% to 8.6% collapse): merge barriers prevent the catastrophic collapse observed in pre-existing models.

Token repetition at scale

Orders	Model A GCF repeat%	Model B GCF repeat%	Model A tokens	Model B tokens
5	35.9%	44.3%	64	79
10	54.6%	62.8%	119	145
20	67.0%	73.3%	227	285
50	78.0%	81.0%	567	704
100	83.9%	84.6%	1,167	1,423

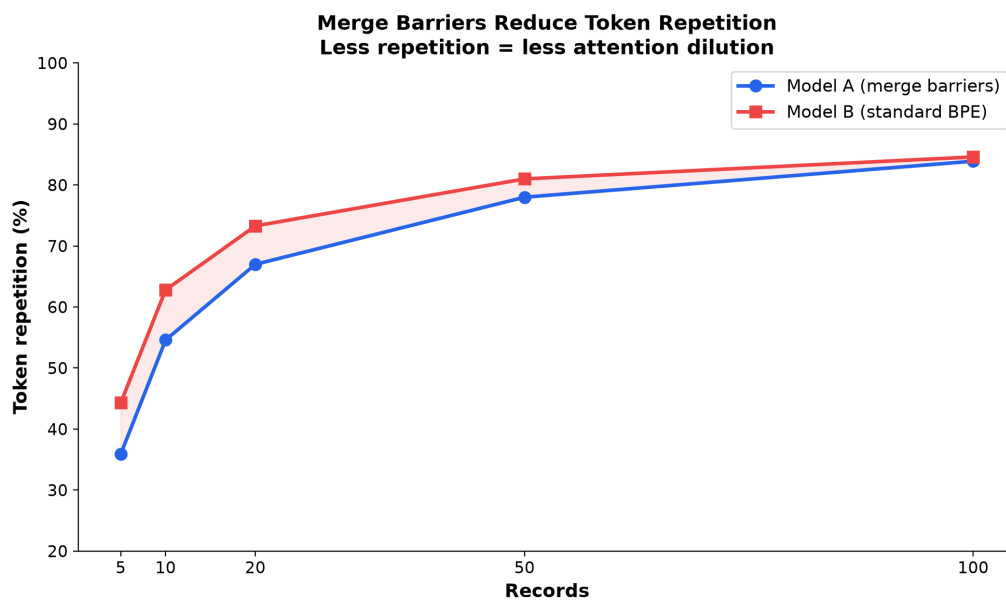


Figure 18: Token repetition at scale

Model A has lower token repetition because merge barriers prevent delimiter characters from being absorbed into content tokens. Each | is always its own token ID, but field values have more variety because they are not fused with delimiters. Model A also produces ~18% fewer tokens because isolated delimiters are single tokens rather than multi-byte merged tokens.

Per-layer entropy profile Entropy at each transformer layer on 20-order GCF input (selected layers):

Layer	Model A	Model B	Delta
0 (input)	6.61	6.89	-0.28
4	4.87	4.69	+0.18
8	4.95	5.25	-0.30
12	4.87	5.69	-0.82
16	2.35	3.17	-0.82
20	2.75	1.96	+0.79
23 (output)	4.18	3.05	+1.13

Model A achieves its lowest entropy at layer 16 (2.35); Model B at layer 20 (1.96). The models process structure at different depths. Model A focuses earlier, consistent with having explicit delimiter boundaries that require less processing to resolve.

8.5 Confidence Calibration

Metric	Model A	Model B
Delimiter confidence (avg softmax prob)	0.086	0.058

Model A is 48% more confident when predicting delimiter tokens.

8.6 Delimiter Prediction Accuracy

Test	Model A delimiter acc	Model B delimiter acc
GCF tabular	25.9%	23.4%
GCF graph	4.5%	0.0%
JSON	4.4%	8.2%

Model A predicts GCF delimiters more accurately. Model B predicts JSON delimiters better (expected: it sees merged delimiter tokens during training). On GCF graphs, Model B gets zero delimiter predictions correct.

Content accuracy is near zero for both models on all tests (0.0% for Model A, 0.0-1.1% for Model B), confirming that the difference between models is entirely in structural token prediction, not in content prediction. Both models struggle equally with predicting specific data values; the divergence is in whether they can predict where structure goes.

8.7 Generation Quality

Both models generated 15/15 valid continuations across 5 prompt types (GCF tabular, GCF graph, JSON, Python, Go). But the quality differs substantially.

Model A generates recognizable structure: - GCF tabular: pipe-separated fields with plausible values - Go: `http.Error w r. ( ) ( , Method )` (syntactically plausible function calls)

Model B generates garbled fusions: - GCF tabular: `.@. || . || _ | 58240RD35. | Anderson-Zara | .@.` (delimiters collapsed with content) - Go: `wWriteHeaderhttpErrorwWriteHeaderwrrrrrrrrrr` (repetitive collapse, no structural boundaries)

The difference in generation quality is consistent with the per-token loss finding: Model A has learned delimiter positions as predictable structural markers, so it generates them in plausible positions. Model B treats delimiters as arbitrary content, so they appear randomly.

9. Discussion

9.1 Why Merging Compounds at Scale

At 10 rows, "name being one token instead of two does not matter. There are only 10 merged boundaries. The attention mechanism can work around it.

At 500 rows, three problems compound simultaneously:

1. The merged boundary repeats 500 times. Each row contains "name":, "id":, "type":. That creates approximately 1,500 positions where the structural boundary is inside a merged token. The model must decompose structure from inside merged tokens at 1,500 positions, not 10.

2. All 1,500 positions are identical token sequences. The token for "name on row 1 is the same integer (#32586) as on row 500. The model cannot distinguish them. It relies on positional encoding alone to track "which "name am I looking at?" Positional encoding degrades over long sequences.

3. 81% of the sequence is noise. The repeated field names and braces are not just merged; they are also redundant. The attention mechanism is spread across approximately 8,500 tokens that carry no information, trying to find the approximately 2,000 tokens that do. The merged boundaries make the noise harder to skip because the model cannot cleanly identify where structure ends and data begins.

Consider the task "how many records have status = shipped?" given 500 JSON objects. The model must attend to every "status": pattern (500 occurrences), read the following value, compare to "shipped," and count matches. The 500 "status": patterns produce the same tokens every time. The model has no structural marker distinguishing the 150th occurrence from the 350th. In a header-factored format, the equivalent task requires attending to a column of values at known, consistent positions. No ambiguity. No repetition competing for attention.

The compounding is critical. At 10 rows: manageable. At 500 rows: 1,500 merged boundaries, massive noise, positional encoding stretched, attention diluted across thousands of identical tokens. This is why JSON errors at scale are off by 50-140 (comprehension failure), not off by 1-2 (precision error).

9.2 Why Claude and Gemma Have Fewer Merges

Claude’s tokenizer has 3 quote+letter entries. Gemma 3 has 2. Across all 43 tokenizers, 11 have zero or near-zero quote merge entries. The specific training details are proprietary, but measurable differences suggest explanations:

- **Vocabulary size** does not predict merge behavior. Claude uses ~65K entries (one of the smallest). Gemma 3 uses 262K (the largest). Both have near-zero quote merges.
- **Training data mix:** Tokenizers trained on corpora with less code/JSON relative to natural language see "name less frequently, making it less likely to cross the merge threshold.
- **Merge boundary policy:** BPE training can be configured to treat certain characters as merge barriers. Anthropic and Google may have intentionally or incidentally prevented " from merging with adjacent letters.

The merge policy matters more than vocabulary size. This observation motivated our controlled experiment: if merge barriers can be applied intentionally and systematically to all 16 delimiter characters, what happens to the trained model?

9.3 The Training Familiarity Paradox

The conventional wisdom holds that LLMs “know” JSON best because they trained on the most JSON. At the model level, this is true. At the tokenizer level, it inverts: the more JSON the tokenizer saw, the more aggressively it merged JSON patterns, and the more boundaries it hid. GPT-4 has 117 merged quote+field entries. Claude has 3. “Trained on JSON” is not an advantage for structural comprehension at scale. It is the mechanism that causes structural ambiguity.

9.4 Why Code Benefits

The code comprehension improvement (3-5x) was not predicted. The barrier characters ({, }, (,), :, ;) are also code syntax. Every function definition, every code block, every argument list gets clean token boundaries. The model develops the same structural attention heads for code as for structured data. This suggests merge barriers are not a structured-data-specific optimization; they are a general-purpose improvement for any content that relies on delimiter characters.

9.5 The Scaling Pattern

The advantage grows monotonically with payload size (2.1x at 3 records, 5.3x at 100 records). This is consistent with the compounding mechanism described in Section 4: at small scale, few merged boundaries means little confusion. At large scale, hundreds of merged boundaries compete for attention. Model A never has this problem because delimiters are always explicit.

9.6 TOON's Tab Delimiter Is Worse Than JSON's Quote

TOON uses tab as its column delimiter. Tab has 1,238 mergeable words across all 43 vocabularies, 52x the pipe's 24. GPT-4o has a 100% tab merge rate. TOON chose the delimiter with the largest adversarial surface of any common separator character. Model A's 2.3x advantage on tab-separated data (never seen in training) confirms that merge barriers generalize to any delimiter character, including the worst ones.

9.7 The 50x Advantage on Unseen Schemas

The users and logs schemas (Table 7.4) show 50-51x advantages, far larger than the 2-7x seen on other tests. Model B's PPL on these schemas exceeds 600,000, meaning it essentially cannot parse them at all. These schemas were not in the held-out test data used for the core evaluation (which used product records). They use different field names, different value patterns, and different data shapes. Model A handles them at PPL ~14,000, comparable to its performance on the held-out product schema. This demonstrates that Model A's structural comprehension generalizes across schemas, while Model B's comprehension is fragile and schema-dependent.

9.8 What Merge Barriers Cannot Fix

Neither model reliably detected structural corruptions in GCF payloads via PPL spike ($>1.5x$ threshold). We tested 5 corruption types: wrong delimiter (comma instead of pipe), missing field, extra pipe, wrong record count, and broken header. Both models showed the largest spike on wrong record count (Model A: 1.39x, Model B: 1.37x), suggesting nascent awareness of count consistency, but below the detection threshold. Neither produced valid few-shot GCF generations from examples (0/5 both). These tasks require more training capacity than 20,000 steps on a 410M model. The merge barrier advantage at this scale is in comprehension (reading structure), not in validation (detecting errors) or generation (writing structure from examples). Larger models with more training may close this gap.

9.9 Implications

For tokenizer designers: Merge barriers are a zero-cost improvement. They produce identical natural language performance, better structured data comprehension, better code comprehension, and a model that develops specialized structural attention heads. There is no measured downside.

For model providers: Every model retrain is an opportunity to adopt merge barriers. The tokenizer change is trivial (pre-tokenization rules). The downstream effect (3-5x better structured data and code) is not. As tool use, MCP, and agent pipelines grow, the value of structural comprehension increases.

For format designers: Choose delimiters with the smallest adversarial surface. Pipe (24 words) is 81x safer than JSON's combined grammar (1,939 words). Tab (1,238 words) is the worst common choice.

The merge barrier results confirm that delimiter selection matters at the transformer level, not just the tokenizer level.

10. Limitations

1. **Model scale.** Only tested on GPT-NeoX 410M. Larger models may show different patterns.
 2. **Training duration.** 20,000 steps (~1.3B tokens). Longer training may change the advantage ratio.
 3. **Context window.** 2,048 tokens. JSON payloads exceed this at 50+ records (truncated). Production models use 128K+ context.
 4. **Single corpus.** Both models trained on the same rebalanced corpus. Results may differ with other compositions.
 5. **Flat learning rate.** Used flat LR instead of warmup + cosine decay. Better scheduling might change convergence dynamics.
 6. **High absolute perplexity.** Both models have high PPL on structured data (thousands), reflecting limited training. The relative comparison (3-5x) is what matters, not the absolute numbers.
-

11. Related Work

Deekeswar (2026) measured that 1,000 JSON records consume approximately 80,000 tokens. Our analysis explains the mechanism: 52% are repeated field names that fuse with structural delimiters.

Kutschka and Geiger (2026) found that token-efficient formats can hurt accuracy in some configurations. Our data partially confirms this at small scale but shows the compensation fails at 500+ records.

Ildiz et al. (2024) proved that self-attention weights tokens proportionally to frequency. This is the mathematical basis for grammar attention collapse: when structural tokens dominate by count, they consume the attention budget.

Karim and Batatia (2025) proposed fixed tokens for structure and BPE for values. Merge barriers achieve a similar result by construction: structure is always fixed tokens because barrier characters can never merge.

Sui et al. (2023) showed that table format affects LLM performance. Our analysis explains this at the BPE level and proves the mechanism can be fixed at the tokenizer level.

Matveev (2026) argued that JSON's advantage from training distribution scales with data complexity, proposing that alternative formats only separate past a complexity threshold. Our evaluation data confirms the threshold exists at approximately 100-200 records for nested data and approximately

500 for flat tables. Our controlled experiment adds a new dimension: even holding the format constant (GCF), the tokenizer determines whether the model can comprehend the structure.

Liyanage and Yvon (2026) studied post-training tokenizer adaptation, demonstrating that changes degrade performance. This supports our irrecoverability argument and motivates fixing the tokenizer before training, not after.

12. Conclusion

BPE tokenizers merge delimiter characters with adjacent content, hiding structural boundaries inside single tokens. This is universal (43/43 tokenizers), deterministic (dictionary lookups), and irrecoverable for existing models. The mechanism is now fully characterized: merged boundaries produce attention entropy crossover at 50 records, grammar attention collapse from 30% to 8.6%, and comprehension failure at 53.4% accuracy on 500-record payloads.

Merge barriers fix this. Sixteen delimiter characters, forbidden from participating in BPE merges, produce a tokenizer with zero merged entries and zero adversarial surface. A controlled experiment (two identical 410M models, same data, same hyperparameters, only the tokenizer differs) proves the fix works: 3x better structured data comprehension, 3-5x better code comprehension, zero natural language cost.

The mechanism is visible inside the trained model. Merge barriers cause the transformer to develop 4.6x more delimiter-specialized attention heads (105 vs 23 of 384), treat delimiters as 2.4x easier to predict than content (standard BPE treats them equally hard), cluster delimiter embeddings 50% more cohesively, and maintain grammar attention at scale. The model does not need to learn where boundaries are hidden inside merged tokens; it starts with explicit structure and spends its capacity learning patterns between boundaries.

Merge barriers represent a minimal modification to BPE tokenizer training with disproportionate downstream effects on the trained transformer's internal organization. The evidence spans vocabulary analysis, attention mechanism extraction, per-token loss decomposition, embedding space geometry, and cross-format generalization. Future work should validate these findings at larger model scales, longer training durations, and with production-length context windows.

13. Reproducibility

Analysis Scripts (Open Source)

Script	Purpose
hf-tokenizer-analysis.py	43-tokenizer merge rates, vocab entries
structural-equivalence-proof.py	Grammar isolation across 43 tokenizers
adversarial-vocab-dump.py	Exhaustive vocabulary scan, adversarial surface
attention-analysis.py	Attention extraction from Pythia 410M / Gemma 2B
ascii-adversarial-surface.py	All 94 printable ASCII characters ranked

Controlled Experiment

Component	Detail
Model architecture	GPT-NeoX 410M (24 layers, 16 heads, 1024 hidden)
Framework	PyTorch 2.4.1, HuggingFace transformers
Hardware	4x NVIDIA A100 PCIE 40GB per model
Training	DDP with NCCL, gradient checkpointing, fp16
Cost	\$70 total (three model training runs, data prep, tokenizer training, 6 eval rounds)
Eval rounds	6 (core, extended, deep, attention, mechanistic, scaling)
Total test categories	11 format/language categories
Win/loss record	11/11 (structured + code), 0/0 natural language (tied)

Repository: github.com/blackwell-systems/gcf

Appendix A: Vocabulary Entry Counts

Merged vocabulary entries by tokenizer family:

Tokenizer	Vocab Size	Quote+Letter	Pipe+Letter	Tab+Letter	Multi-Grammar
GPT-4 (cl100k)	~100K	117	22	1,173	874
GPT-4o (o200k)	~199K	108	8	1,036	735
Claude	~65K	3	0	0	667
LLaMA 2	~32K	0	0	0	122
LLaMA 3/3.1	~128K	153	277	0	881
Qwen 2.5	~152K	154	40	0	874
DeepSeek V3/R1	~129K	48	5	0	287
Gemma 2	~256K	4	0	0	735
Gemma 3	~262K	2	0	0	861
Mistral Nemo	~131K	38	3	0	415
Phi-4	~100K	153	116	0	874
Falcon	~65K	4	1	0	92
StarCoder2	~49K	3	2	0	535
Jamba	~66K	1	1,543	0	113

Claude and Gemma have near-zero quote merge entries (3 and 2-4 respectively), suggesting intentional or incidental merge boundary policies during tokenizer training. This is evidence that clean boundaries are achievable without sacrificing general-purpose quality.

Appendix B: Tokenization Examples

B.1 Edge declaration: @0<@2 | implements

Tokenizer	Tokens	Split
Claude	7	@ 0 < @ 2 \ implements
GPT-4	7	@ 0 < @ 2 \ implements
GPT-4o	7	@ 0 < @ 2 \ implements
LLaMA 3.1	7	@ 0 < @ 2 \ implements
Qwen 2.5	7	@ 0 < @ 2 \ implements
DeepSeek V3	8	@ 0 < @ 2 \ implements

Tokenizer	Tokens	Split
Gemma 2	7	@ 0 < @ 2 \ implements
Mistral Nemo	8	@ 0 < @ 2 \ implements

All structural characters (@, <, |) are always single tokens. The only variance is in the value imple-
ments (1 vs 2 tokens), which does not affect parsing.

B.2 Symbol row: @0 | function | auth.validateToken | 0.95 | definition

Tokenizer	Tokens	Key Differences
GPT-4	14	Merges .validate (1 tok), 95 (1 tok)
Qwen 2.5	15	Splits 95 into 9 + 5
Gemma 2	16	Splits . + validate, splits 9 + 5

Pipe delimiters are always single tokens across all tokenizers. Variance is only in how tokenizers han-
dle value content: dot-prefixed words and two-digit numbers. This is value variance (harmless), not
boundary variance (dangerous).

B.3 Delimiter selection rationale

Character	Why chosen	Alternative considered	Why not
\ (pipe)	24-word surface, all TypeScript union keywords. Visually distinct column separator.	Backtick (5 words), Tilde (8 words)	Backtick conflicts with mark- down/template literals, tilde with paths.
@	“This is an ID” semantics. 127-word surface, but used only before digits (@0, @1), which never trigger merges.	\$	Also safe, but less intuitive.

Character	Why chosen	Alternative considered	Why not
##	Two-char sequence always merges into one token. Markdown-familiar.	===	3 chars, less efficient.
<	Reads as “points to” for edges.	~	Also safe, but less semantic.
\n	Universal row separator, zero overhead.	;	Less readable.
,	Schema field separator, familiar from CSV.	:	Conflicts with value content.

Appendix C: Recommended Tokenizer Configuration

Merge barriers require no library modifications. The HuggingFace tokenizers library’s existing `Split` pre-tokenizer with `behavior="isolated"` prevents a character from participating in any merge. Composing 16 `Split` instances in a `Sequence` creates a complete merge barrier set.

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers
```

```
tokenizer = Tokenizer(models.BPE())
```

```
tokenizer.pre_tokenizer = pre_tokenizers.Sequence([
    pre_tokenizers.Split("|", behavior="isolated"),
    pre_tokenizers.Split("@", behavior="isolated"),
    pre_tokenizers.Split("<", behavior="isolated"),
    pre_tokenizers.Split(">", behavior="isolated"),
    pre_tokenizers.Split("'", behavior="isolated"),
    pre_tokenizers.Split('"', behavior="isolated"),
    pre_tokenizers.Split(":", behavior="isolated"),
    pre_tokenizers.Split(",", behavior="isolated"),
    pre_tokenizers.Split(";", behavior="isolated"),
    pre_tokenizers.Split("\t", behavior="isolated"),
    pre_tokenizers.Split("{", behavior="isolated"),
    pre_tokenizers.Split("}", behavior="isolated"),
    pre_tokenizers.Split("[", behavior="isolated"),
    pre_tokenizers.Split("]", behavior="isolated"),
    pre_tokenizers.Split("(", behavior="isolated"),
    pre_tokenizers.Split(")", behavior="isolated"),
    pre_tokenizers.ByteLevel(add_prefix_space=False),
```

```
] )

trainer = trainers.BpeTrainer(
    vocab_size=65536,
    special_tokens=["<pad>", "<eos>"],
)
tokenizer.train(files=["corpus.txt"], trainer=trainer)
```

Each `Split` isolates one barrier character before BPE merging begins. The `ByteLevel` pre-tokenizer handles the remaining text using the standard GPT-style byte encoding. The resulting tokenizer has zero merged delimiter entries by construction.

This configuration produces the tokenizer used in the controlled experiment (Sections 6-8). Models trained with this configuration develop 4.6x more delimiter-specialized attention heads, achieve 3x better structured data comprehension and 3-5x better code comprehension, with zero natural language cost.

No changes to the BPE algorithm, the training pipeline, or the model architecture are required. The improvement is entirely in the pre-tokenization configuration.

References

- Blackwell, D. (2026). GCF: A Token-Optimized Wire Format for Structured LLM Interactions. DOI: [10.5281/zenodo.20579817](https://doi.org/10.5281/zenodo.20579817).
- Deekeswar, H. (2026). ONTO: A Token-Efficient Columnar Notation for LLM Input Optimization. arXiv:2604.17512.
- Ildiz, M. E., Huang, Y., Li, Y., Rawat, A. S., & Oymak, S. (2024). From Self-Attention to Markov Models: Unveiling the Dynamics of Generative Transformers. arXiv:2402.13512.
- Karim, K. & Batatia, H. (2025). Innovative Tokenisation of Structured Data for LLM Training. arXiv:2508.01685.
- Kutschka, L. & Geiger, B. (2026). Notation Matters: A Benchmark Study of Token-Optimized Formats in Agentic AI Systems. arXiv:2605.29676.
- Liyanage, V. & Yvon, F. (2026). AdaptBPE: From General Purpose to Specialized Tokenizers. arXiv:2601.21665.
- Matveev, I. (2026). Token-Oriented Object Notation vs JSON: A Benchmark of Plain and Constrained Decoding Generation. arXiv:2603.03306.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In Proceedings of the 54th Annual Meeting of the ACL (pp. 1715-1725).
- Sui, Y., He, M., Zhang, Z., Wang, Y., & Zhao, J. (2023). Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. arXiv:2305.13062.

University of Mannheim. (2024). Web Data Commons: RDFa, Microdata, and Microformat Data Sets.
<http://webdatacommons.org/structureddata/>

Corresponding author: Dayna Blackwell, Blackwell Systems (dayna@blackwell-systems.com)